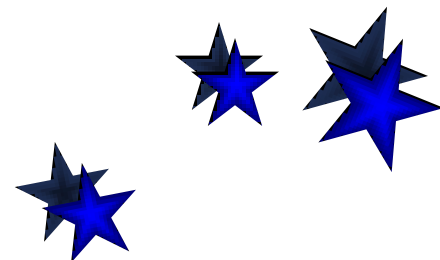


1.3 算法的数学基础

- 求解递归方程的方法
 - (1) 迭代法（递推法）
 - (2) 生成函数求解
 - (3) 特征方程求解
 - (4) 递归树
 - (5) 主方法

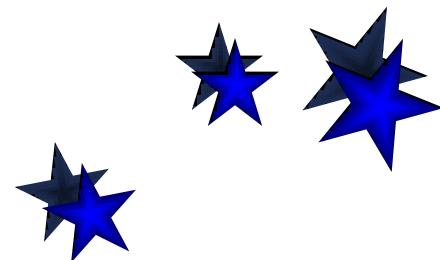


1.3 算法的数学基础

- 迭代法（递推法）求解递归方程

- 直接从递归方程出发，一层一层地往前递推，直到最前面的初始条件为止，就得到了问题的解。

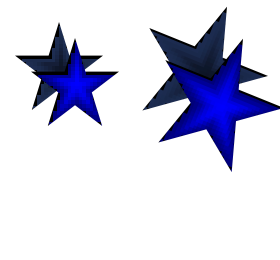
$$(1)F(n) = \begin{cases} F(n-1) + 1 & (n > 1) \\ 1 & (n = 1) \end{cases}$$



1.3 算法的数学基础

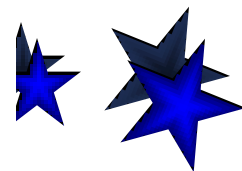
$$\begin{aligned}T(n) &= 2 \, \underline{T(n-1)} + 1 \\&= 2 \, [\, \underline{2T(n-2)} + 1 \,] + 1 \\&= 2^2 T(n-2) + 2 + 1 \\&= \dots \\&= 2^{n-1} T(1) + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 \\&= 2^{n-1} + 2^{n-1} - 1 \\&= 2^n - 1\end{aligned}$$

代入初值
求和



1.3 算法的数学基础

$$\begin{aligned}W(n) &= W(n-1) + n-1 \\&= [W(n-2) + n-2] + n-1 \\&= W(n-2) + n-2 + n-1 \\&= \dots \\&= W(1) + 1 + 2 + \dots + (n-2) + (n-1) \\&= 1 + 2 + \dots + (n-2) + (n-1) \\&= n(n-1)/2\end{aligned}$$

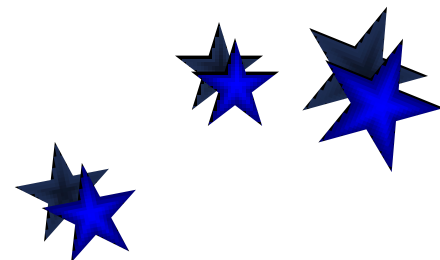


1.3 算法的数学基础

$$(4)F(n) = \begin{cases} 2F(n/2) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

□ 换元迭代

- 将对 n 的递推式换成对其他变元 k 的递推式
- 对 k 直接迭代
- 将解 (关于 k 的函数) 转换成关于 n 的函数



1.3 算法的数学基础

$$(4)F(n)=\begin{cases} 2F(n/2)+n-1 & (n>1) \\ 0 & (n=1) \end{cases}$$

假设 $n=2^k$, 递推方程如下:

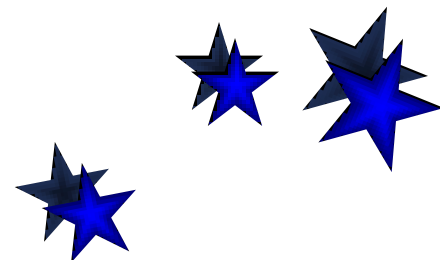
$$F(n)=2F(n/2)+n-1 \quad (n>1)$$

$$F(1)=0 \quad (n=1)$$

换元:

$$F(2^k) = 2F(2^{k-1}) + 2^k - 1$$

$$F(0) = 0$$



1.3 算法的数学基础

$$W(n) = 2W(2^{k-1}) + 2^k - 1$$

$$= 2[2W(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1$$

$$= 2^2 W(2^{k-2}) + 2^k - 2 + 2^k - 1$$

$$= 2^2 [2W(2^{k-3}) + 2^{k-2} - 1] + 2^k - 2 + 2^k - 1$$

$$= \dots$$

$$= 2^k W(1) + k2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1)$$

$$= k2^k - 2^k + 1$$

$$= n \log n - n + 1$$

换元，
对k迭代



1.3 算法的数学基础

解的正确性——归纳法验证

证明:下述递推方程的解是 $W(n)=n(n-1)/2$

$$W(n)=W(n-1)+n-1$$

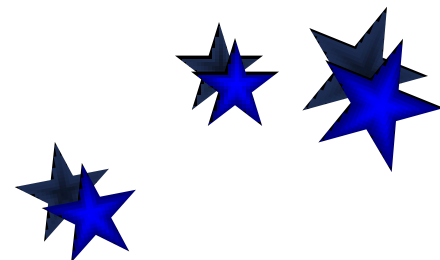
$$W(1)=0$$

方法: 数学归纳法

证 $n=1$, $W(1)=1*(1-1)/2 = 0$,

假设对于 n , 解满足方程, 则

$$\begin{aligned} W(n) &= W(n-1)+n-1 = (n-2)(n-1)/2 + n - 1 \\ &= (n^2-n)/2=n(n-1)/2 \end{aligned}$$



1.3 算法的数学基础

证明:下述递推方程的解是 $W(n)=n\log n-n+1$

$$W(n)=2W(n/2)+n-1$$

$$W(1)=0$$

方法: 数学归纳法

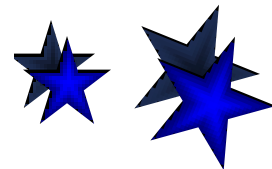
证 $n=1$, $W(1)=1*\log 1-1+1=0$,

假设对于 n , 解满足方程, 则

$$W(n) = 2W(n/2)+n-1 = 2\left(\frac{n}{2}\log\frac{n}{2}-\frac{n}{2}+1\right)+n-1$$

$$= n\log\frac{n}{2}-n+2+n-1$$

$$= n(\log n - 1) + 1 = n\log n - n + 1$$



1.3 算法的数学基础

快速排序平均时间分析

(1) 快速排序的时间构成:

分划的时间(找元素 a_1 最终位置的时间)

+ 处理子问题1的时间

+ 处理子问题2的时间

(2) 元素 a_1 在数组的任何位置都有可能, 因此概率为 $1/n$

a_1 排序后的位置	子问题规模	子问题2规模	划分的比较次数
1	0	$n-1$	$n-1$
2	1	$n-2$	$n-1$
.....			
n	$n-1$	0	$n-1$



1.3 算法的数学基础

快速排序平均时间分析

差消化简后迭代—高阶化简为一阶

$$\begin{cases} T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + cn, & n \geq 2 \\ T(1) = 0 \end{cases}$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2$$

$$nT(n) = (n+1)T(n-1) + 2cn - c$$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2cn - c}{n(n+1)} = \dots$$

$$= 2c \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} + \frac{T(1)}{2} \right] - O\left(\frac{1}{n}\right) = \Theta(\log n)$$

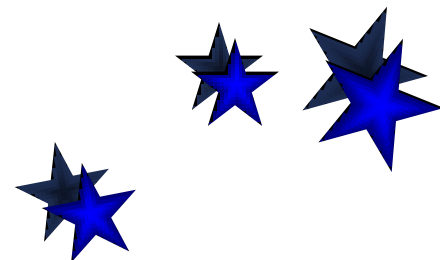
$$T(n) = \Theta(n \log n)$$

1.3 算法的数学基础

- 递归树求解递归方程

- ◆ 递归树的概念

- 递归树是迭代计算的模型
 - 递归树的生成过程与迭代过程一致
 - 递归树上所有项恰好是迭代之后产生和式中的项
 - 对递归树上的项求和就是迭代后方程的解



1.3 算法的数学基础

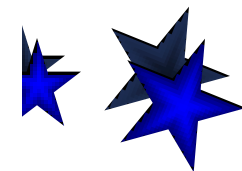
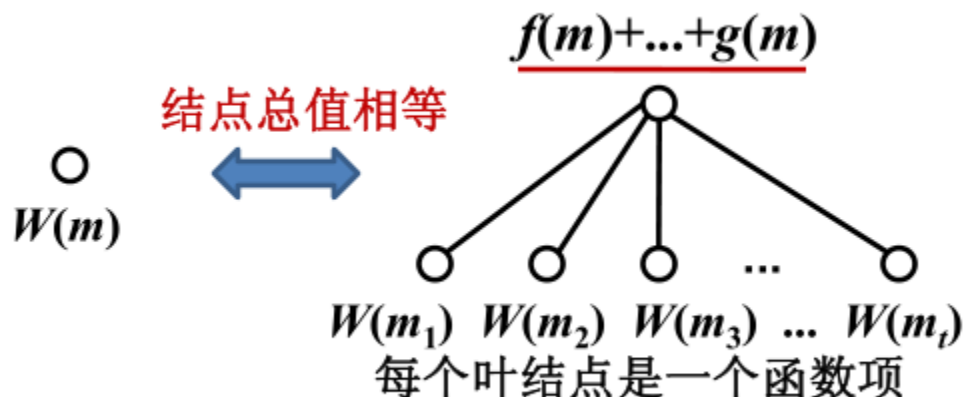
◆ 迭代在递归树中的表示

如果递归树上某结点标记为 $W(m)$

$$W(m) = W(m_1) + \dots + W(m_t)$$

$$+ \underline{f(m) + \dots + g(m)}, \quad m_1, \dots, m_t < m$$

其中 $W(m_1), \dots, W(m_t)$ 称为函数项。



1.3 算法的数学基础

- 递归树求解递归方程

- ◆ 递归树的生成规则

- 初始，递归树只有根结点, 其值为 $W(n)$

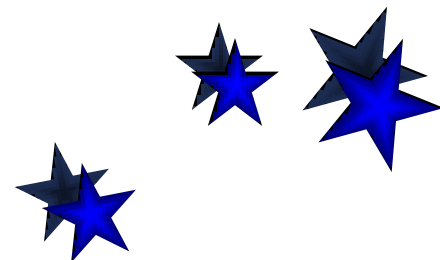
- 不断继续下述过程:

- 将函数项叶结点的迭代式 $W(m)$ 表示成二层子树

- 用该子树替换该叶结点

- 继续递归树的生成，直到树中无函数项 (只有初值)为止

-

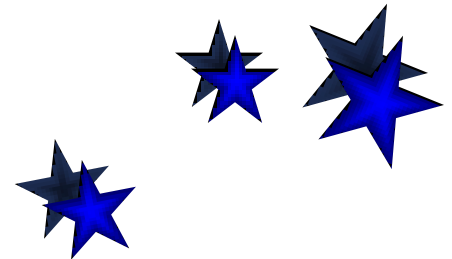
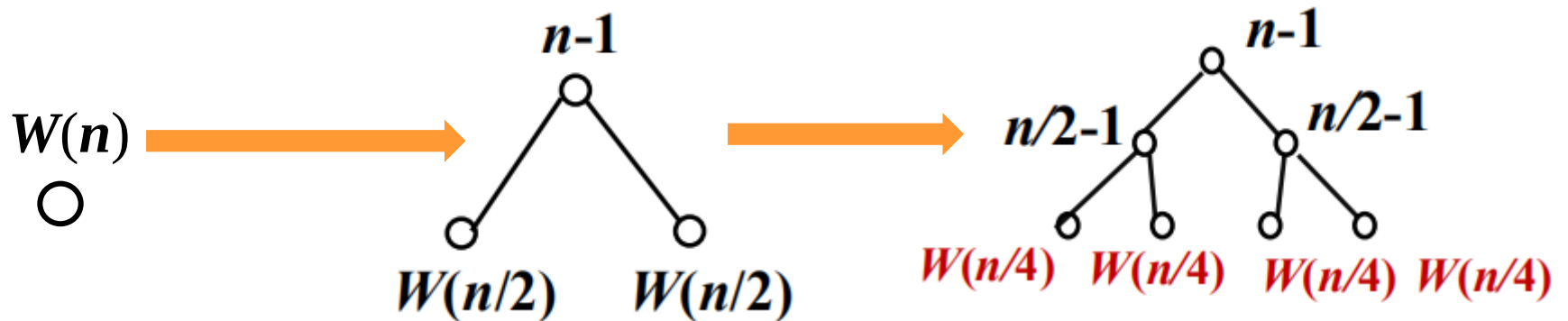


1.3 算法的数学基础

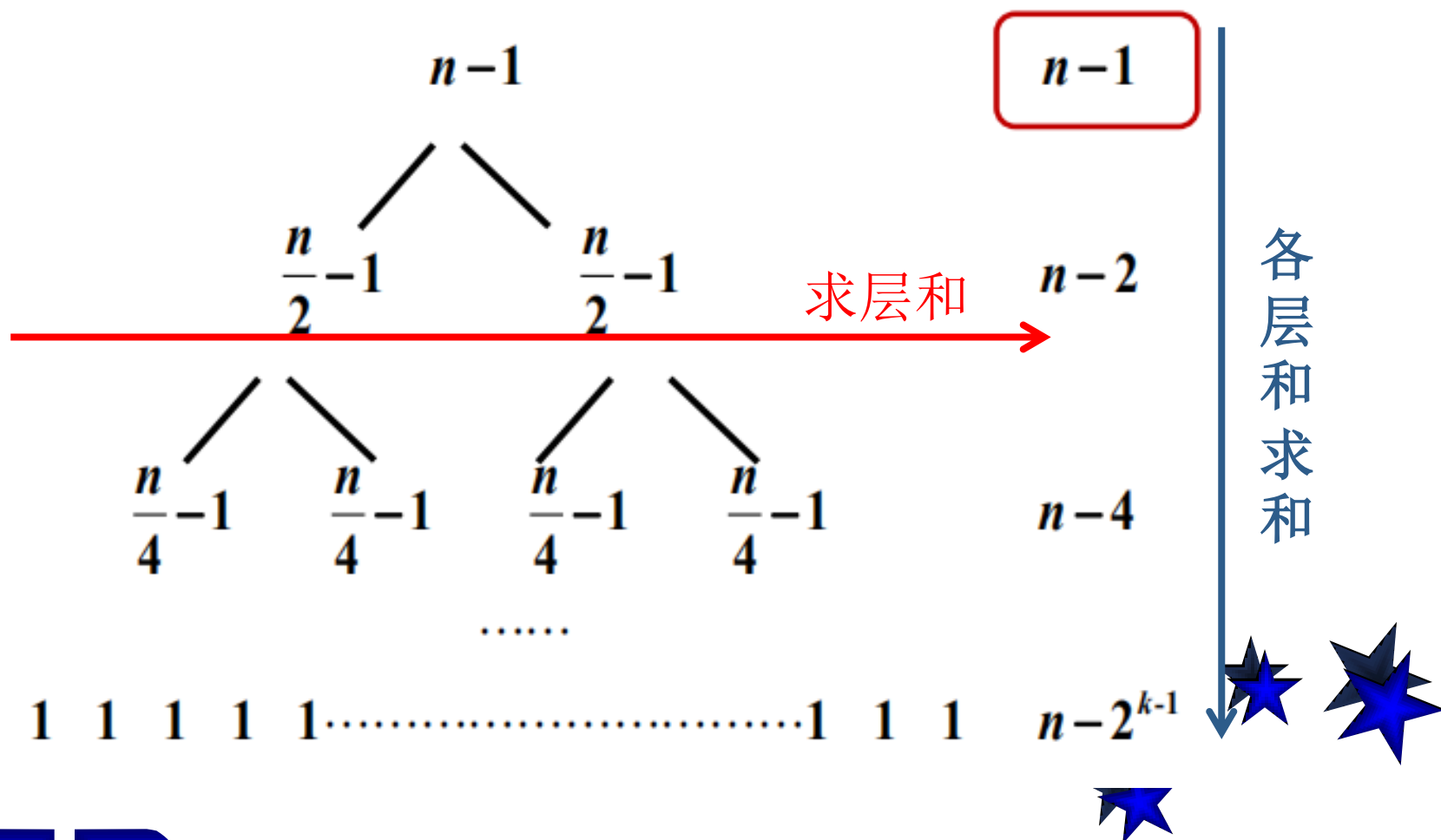
- 递归树求解递归方程

二分归并排序

$$W(n) = 2W(n/2) + \underline{n-1}$$



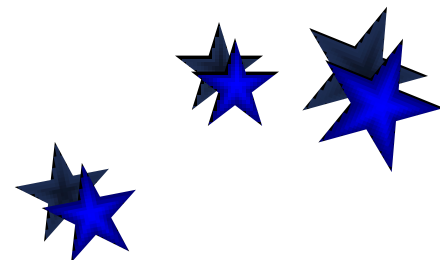
1.3 算法的数学基础



1.3 算法的数学基础

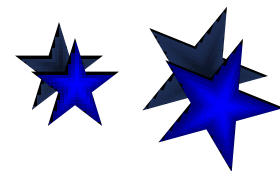
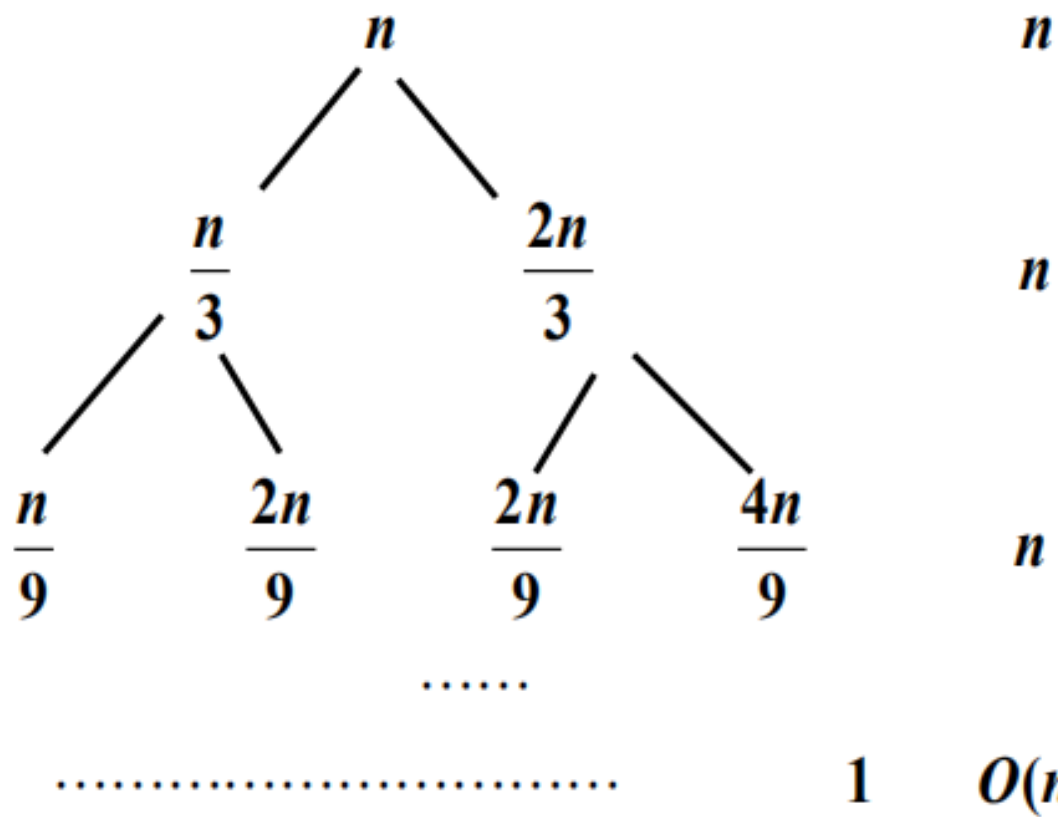
$$\begin{aligned} W(n) &= 2W(n/2) + n - 1, \quad n = 2^k, \\ W(1) &= 0 \end{aligned}$$

$$\begin{aligned} W(n) &= n - 1 + n - 2 + \dots + n - 2^{k-1} \\ &= kn - (2^k - 1) \\ &= n \log n - n + 1 \end{aligned}$$



1.3 算法的数学基础

求解方程: $T(n) = T(n/3) + T(2n/3) + n$



1.3 算法的数学基础

可见每层的值都为 n ，从根到叶节点的最长路径是：

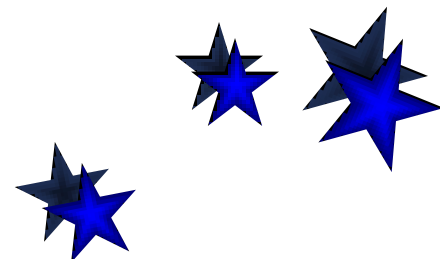
$$n \rightarrow \frac{2}{3}n \rightarrow \left(\frac{2}{3}\right)^2 n \rightarrow \dots \rightarrow 1$$

因为最后递归的停止是在 $(2/3)^k n == 1$. 则

$$k = \log_{3/2} n$$

$$T(n) \leq \sum_{i=0}^k n = (k+1)n = n(\log_{3/2} n + 1)$$

即 $T(n) = O(n \log n)$



1.3 算法的数学基础

● 主定理（Master Method）求解递归方程

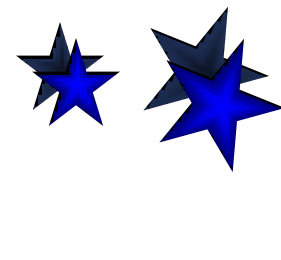
定理：设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为非负整数，且

$$T(n) = aT(n/b) + f(n) \text{ —分治策略}$$

则有以下结果：

1. 若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$
3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对某个常数 $c < 1$ 和所有充分大的 n 有 $a f(n/b) \leq c f(n)$, 那么

$$T(n) = \Theta(f(n))$$



1.3 算法的数学基础

例9 求解递推方程 $T(n) = 9T(n/3) + n$

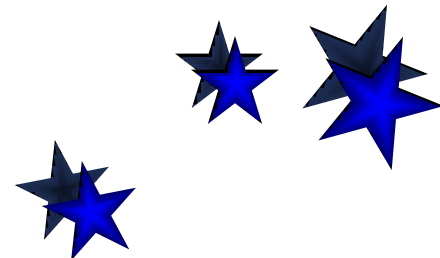
解：上述递推方程中的 $a = 9, b = 3, f(n) = n$, 那么

$$n^{\log_3 9} = n^2, \quad f(n) = O(n^{\log_3 9 - 1}),$$

相当于主定理的第一种情况，其中 $\varepsilon = 1$. 根据定理得到

$$T(n) = \Theta(n^2)$$

练习： $T(n) = 5T(n/2) + \Theta(n^2)$



1.3 算法的数学基础

例10 求解递推方程 $T(n) = T(2n/3) + 1$

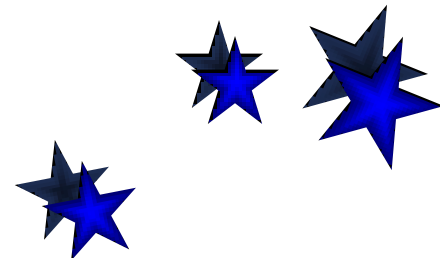
解 上述递推方程中的 $a=1, b=3/2, f(n)=1$, 那么

$$n^{\log_{3/2} 1} = n^0 = 1, \quad f(n) = 1$$

相当于主定理的第二种情况. 根据定理得到 .

$$T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

练习: $T(n) = 2T(n/2) + \Theta(n)$



1.3 算法的数学基础

练习: $T(n)=5T(n/2)+\Theta(n^3)$

例11 求解递推方程

$$T(n) = 3T(n/4) + n \log n$$

解 上述递推方程中的 $a=3$, $b=4$, $f(n)=n \log n$, 那么

$$n \log n = \Omega(n^{\log_4 3 + \varepsilon}) = \Omega(n^{0.793 + \varepsilon}),$$

$$\varepsilon \approx 0.2$$

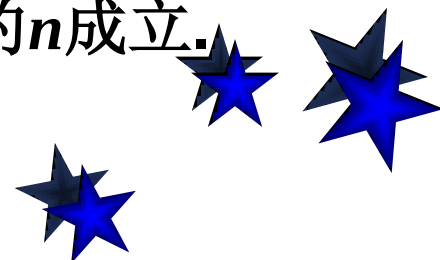
此外, 要使 $a f(n/b) \leq c f(n)$ 成立, 代入 $f(n)=n \log n$, 得到

$$\frac{3n}{4} \log \frac{n}{4} \leq c n \log n$$

显然只要 $c \geq 3/4$, 上述不等式就可以对充分大的 n 成立.

相当于主定理的第三种情况. 因此有

$$T(n) = \Theta(f(n)) = \Theta(n \log n)$$



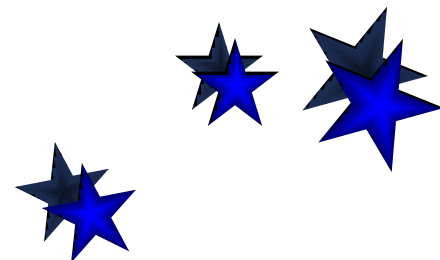
1.3 算法的数学基础

不能使用主定理的例子

例12 求解 $T(n) = 2T(n/2) + n \log n$

$a=b=2$, $n^{\log_2 2} = n$, $f(n) = n \log n$

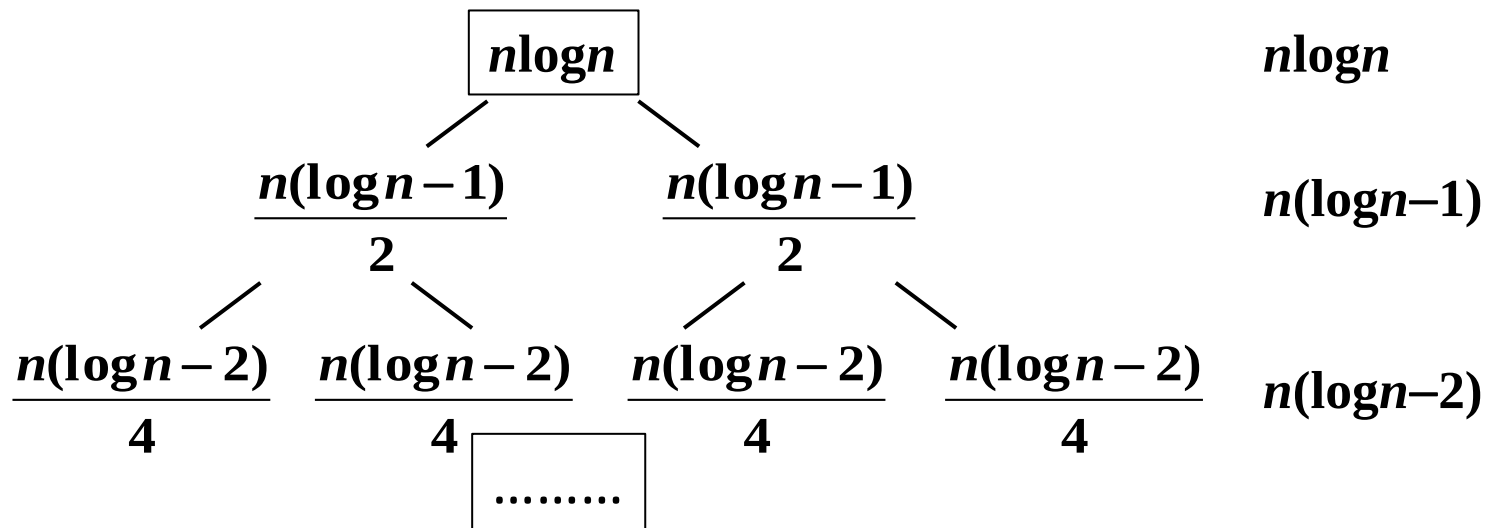
不存在 $\varepsilon > 0$ 使得 $n \log n = \Omega(n^{1+\varepsilon})$ 成立,



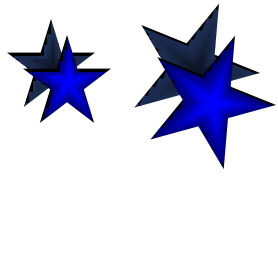
1.3 算法的数学基础

$$T(n) = 2T(n/2) + n \log n$$

$$n^{\log_2 2} = n$$



$$\begin{aligned} T(n) &= n \log n + n(\log n - 1) + n(\log n - 2) + \dots + n(\log n - k + 1) \\ &= (n \log n) \log n - n(1 + 2 + \dots + k - 1) \\ &= n \log^2 n - nk(k - 1)/2 = O(n \log^2 n) \end{aligned}$$

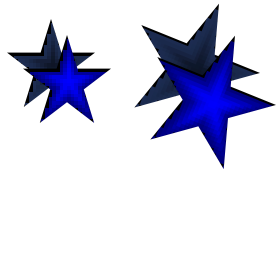


补充：算法空间复杂度的分析

- 一个算法的存储量包括形参所占空间和临时变量所占空间。在对算法进行存储空间分析时，只考察临时变量所占空间。
- 所以，空间复杂度是对一个算法在运行过程中临时占用的存储空间大小的量度，一般也作为问题规模 n 的函数，以数量级形式给出，记作：

$$S(n)=O(g(n))、\Omega(g(n))\text{或}\Theta(g(n))$$

- 其中渐进符号的含义与时间复杂度中的含义相同

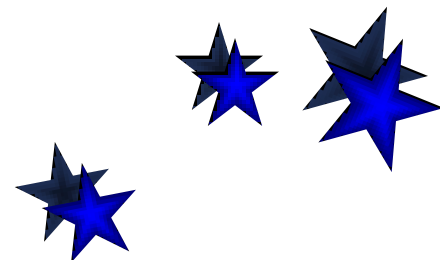


算法空间复杂度的分析

```
int max(int a[], int n)
{ int i, maxi=0;
  for (i=1;i<=n;i++)
    if (a[i]>a[maxi])
      maxi=i;
  return a[maxi];
}
```

函数体内分配的变量空间为临时空间，不计形参占用的空间，这里的仅计*i*、*maxi*变量的空间，其空间复杂度为 $O(1)$ 。

算法中临时空间为变量*i*、*maxi*占用的空间



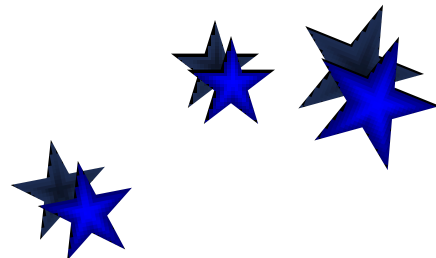
算法空间复杂度的分析

- 为什么算法占用的空间只考虑临时空间，而不必考虑形参的空间呢？这是因为形参的空间会在调用该算法的算法中考虑，例如，以下maxfun算法调用max算法：

```
void maxfun()  
{   int b[]={1,2,3,4,5},n=5;  
    printf("Max=%d\n",max(b,n));  
}
```

```
int max(int a[], int n)  
{   int i, maxi=0;  
    for (i=1;i<=n;i++)  
        if (a[i]>a[maxi])  
            maxi=i;  
    return a[maxi];  
}
```

maxfun算法中为**b**数组分配了相应的内存空间，其空间复杂度为 $O(n)$ ，如果在max算法中再考虑形参**a**的空间，这样重复计算了占用的空间。



算法空间复杂度的分析

- 算法空间复杂度的分析方法与前面介绍的时间复杂度分析方法相似。

【例】分析下面算法的空间复杂度。

```
void func(int n)
{   int i=1,k=100;
    while (i<=n)
    {   k++;
        i+=2;
    }
}
```

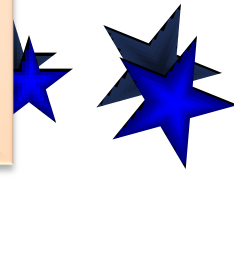
解：该算法是一个非递归算法，其中只临时分配了 i 、 k 两个变量的空间，它与问题规模 n 无关，所以其空间复杂度均为 $O(1)$ 。



算法空间复杂度的分析

【例】有如下递归算法，分析调用`maxelem(a, 0, n-1)`的空间复杂度。

```
int maxelem(int a[],int i,int j)
{
    int mid=(i+j)/2,max1,max2;
    if (i<j)
    {
        max1=maxelem(a,i,mid);
        max2=maxelem(a,mid+1,j);
        return (max1>max2)?max1:max2;
    }
    else return a[i];
}
```



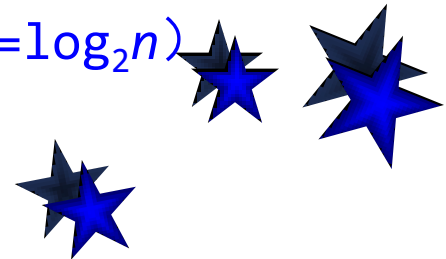
解：执行该递归算法需要多次调用自身，每次调用只临时分配3个整型变量的空间（ $O(1)$ ）。

设调用 $\text{maxelem}(a, 0, n-1)$ 的空间为 $S(n)$ ，有：

```
int maxelem(int a[],int i,int j)
{
    int mid=(i+j)/2,max1,max2;
    if (i<j)
    {
        max1=maxelem(a,i,mid);
        max2=maxelem(a,mid+1,j);
        return (max1>max2)?max1:max2;
    }
    else return a[i];
}
```

$S(n)=O(1)$	当 $n=1$
$S(n)=2S(n/2)+O(1)$	当 $n>1$

$$\begin{aligned}\text{则： } S(n) &= 2S(n/2)+1=2[2S(n/2^2)+1]+1=2^2S(n/2^2)+1+2^1 \\ &= 2^3S(n/2^3)+1+2^1+2^2 \\ &= \dots \\ &= 2^kS(n/2^k)+1+2^1+2^2+\dots+2^{k-1} \quad (\text{设 } n=2^k, \text{ 即 } k=\log_2 n) \\ &= n*1+2^k-1 = 2n-1 = \\ &\quad \mathbf{O(n)}\end{aligned}$$



第一章 小结

■ 有关算法的基本概念

□ 算法设计的任务

(1) 相关举例：调度问题、任务分配问题

(2) 具体任务

- **建模**：对输入参数和解给出形式化或半形式化的描述，分析问题的目标函数、约束函数等。
- **设计算法**：选择**恰当的算法策略**进行设计
- **分析算法的正确性**：是否对所有的实例都得到正确的解（尤其贪心策略一定要证明）
- **分析算法效率**：是否为指数级的（NP难问题），若是，能否为其找到近似多项式阶的解。



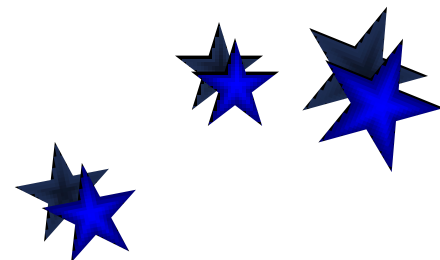
第一章 小结

□ 问题计算复杂度的界定

(1) 相关举例：排序问题

(2) 研究内容

- 对解决同一类问题（排序类、查找类等）的不同算法策略进行效率分析。
- 是否可找到更好的、效率更高的解决该类问题算法



第一章 小结

□ 计算复杂性理论

(1) NP难举例：货郎问题、背包问题、双机调度问题

(2) 研究内容

- 这样的问题有数千个，大量存在于各个应用领域。
- 至今没找到有效算法：现有的算法的运行时间是输入规模的指数或更高阶函数。
- 至今没有人能够证明对于这类问题不存在多项式时间的算法。
- 从是否存在多项式时间算法的角度看，这数千个NP难问题彼此是等价的。若一个NP难问题存在多项式时间，则这些问题的难度处于可有效计算的边界。



第一章 小结

□ 算法及时间复杂度估算

(1) 算法中涉及的基本概念：问题、问题描述、问题实例、算法、算法A解决问题P。

(2) 算法时间复杂度: 针对指定基本运算，计数算法所做该运算的次数，通常是问题输入规模的函数。

- 基本运算的确定
- 问题输入规模的确定

(3) 算法时间复杂度的估算方法

- 根据相同规模的不同输入实例估算：最好、最坏及平均时间复杂度
- 渐进时间复杂度：上界、下界、准确界.....(1.3节)



第一章 小结

□ 算法最好、最坏及平均时间复杂度计算

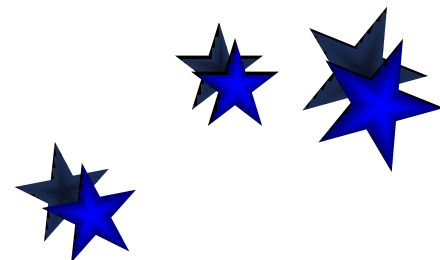
(1) 最好、最坏及平均时间复杂度定义

(2) 相关举例：非递减有序序列的线性检索

- 算法1：未考虑非递减有序特性
- 算法2：考虑非递减有序特性

(3) 研究任务

- 根据具体的问题，给出算法描述
- 分析所设计算法的最好、最坏及平均时间复杂度。



第一章 小结

■ 算法的数学基础

□ 函数的渐近的界

(1) 函数渐近界的符号定义: O 、 Ω 、 Θ 、 o 、 ω

(2) 函数渐近的界的定理1.1、1.2、1.3

(3) 研究内容

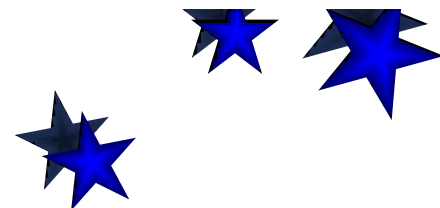
- 分析函数之间满足的关系, 例如 $f(n)$ 表示算法的执行时间, 证明: $f(n)=O(g(n))$, $f(n)=\Theta(g(n))$ ——函数渐近界的符号定义、定理1.1。
- 定理1.2—函数的阶之间的关系具有传递性
- 定理1.3—若每一步的时间复杂度函数的上界都是 $h(n)$ 
，那么该算法的时间复杂度函数可以写作 $O(h(n))$.

第一章 小结

□ 算法时间复杂度估算

(1) 循环类（迭代）算法：算法中基本运算的执行次数的累加和。

- 举例：冒泡排序、选手的竞技淘汰比赛、洗牌程序、二分检索算法
- 求和方法：
 - ✓ 求和公式：等差数列、等比数列、调和级数
 - ✓ 放大法估计累加和的上界
 - ✓ 利用积分求出累加和的渐进的界



第一章 小结

(2) 递归类算法：该类算法的执行时间也表示为递推形式

- 举例：求 $n!$ 、Hanoi塔、直接插入排序、二路归并排序
- 求解递归方程的方法：
 - ✓ 迭代法
 - ✓ 生成函数
 - ✓ 特征方程
 - ✓ 递归树
 - ✓ 主方法

